



Division of Computing Science and Mathematics
Faculty of Natural Sciences
University of Stirling

Invasive Lionfish Identification

**Dissertation submitted in partial fulfilment for the degree of
Master of Science in Big Data**

September 2024

Abstract

This project focuses on the development of a machine learning model with the capacity to receive images of various types of aquatic life and accurately identify if the subject is a lionfish or not.

The issue of invasive lionfish and the threat they pose to native marine ecosystems in the Caribbean has been known since the 1980s. Since then, however, efforts to remove them from said ecosystems have not been as effective as was hoped and both their populations and range in the Atlantic Ocean continue to grow [1]. Complete eradication of lionfish from affected areas is no longer seen as a realistic option so efforts are instead being dedicated to controlling their population and preventing further spread [2]. The most effective method of controlling lionfish populations entails divers (often volunteers) going to affected areas and manually removing them. This method faces some problems, however, such as a lack of sufficient volunteers and some affected areas being dangerous to reach for non-professional divers [3]. There are also concerns that the methods used to hunt lionfish, such as spearfishing, may also be causing inadvertent harm to the native environments that they are trying to protect [4]. By creating a model able to accurately identify lionfish, this project aims to make further progress in the automation of lionfish hunting. The use of technology could help address both the reliance on volunteers and the dangers of deep-water diving while also minimizing the potential harm to the environment from human error [3][4].

Following the CRISP-DM method, the project goal was decided and the necessary data sets, containing images of both lionfish and other aquatic life, were collected. These datasets were inspected and potential issues were addressed, such as slight differences in the number of images and their sizes. Using the cleaned data alongside various Python libraries, a convolutional neural network able to identify lionfish amongst images of aquatic life was developed and tested in both IDLE and Jupyter Notebooks.

After various runs and adjustments, a final model was developed that achieved over 90% accuracy scores. This model is a good first step towards the automation of lionfish hunting and could hopefully pave the way for further advancements in invasive population control.

Attestation

I understand the nature of plagiarism, and I am aware of the University's policy on this.

I certify that this dissertation reports original work by me during my university project except for the following:

- The code discussed in Section 3.4 was referenced from examples found on various coding websites (cited in References) and was also modified and bug tested with assistance from OpenAI (GPT-4).

Signature

Date

Acknowledgements

I would like to thank the staff of University of Stirling, especially Kevin Swingler, Keiller Nogueira, Nora Tanner and Joshua Thorpe for the guidance and assistance they provided during the making of this dissertation.

I would also like to thank my friends and family, who provided me with support and encouragement not just during this project, but throughout the entire Master's degree. Knowing that you all believe in me gives me the motivation to push through. I could not have made it this far without you all.

Table of Contents

Abstract	i
Attestation.....	ii
Acknowledgements.....	iii
Table of Contents.....	iv
1 Introduction	1
1.1 Background and Context	1
1.2 Objectives.....	1
1.3 Achievements.....	2
1.4 Overview of Dissertation.....	2
2 Literature Review	3
3 Methodology.....	4
3.1 Project Planning	4
3.2 Collecting Data	4
3.3 Data Preparation	4
3.4 Model Building	5
4 Conclusion.....	7
4.1 Summary	7
4.2 Evaluation.....	7
4.3 Future Work	7
References.....	9
Appendix	11

1 Introduction

1.1 Background and Context

The existence of invasive lionfish in the Atlantic Ocean has been known since the 1980's. Since then, they have spread to almost every corner of the region and have done considerable damage to native marine populations and the coral reefs. They are voracious predators. Even a single lionfish can consume a considerable number of small reef fish and crustaceans. This is a twofold problem as they are not only eating beneficial herbivores that keep the coral reefs healthy by cleaning them of algae, but also outcompeting local predators who are dependent on the same prey animals. This coupled with their extreme breeding rate and lack of predators in this new environment means that their populations are growing faster than the ecosystem can realistically sustain [1].

Controlling their populations has proven to be a difficult task. Attempts to encourage lionfish hunting through bounty rewards were initially effective but quickly exhausted the funds of wildlife conservation groups as it became abundantly clear that the invasive lionfish populations in the Atlantic have grown too much to realistically eradicate completely. Although lionfish possess venomous spines that greatly discourage most animals from hunting them, they *do* have natural predators in their native regions that also exist in the Atlantic, such as sharks, moray eels and other large predatory fish such as grouper. These Atlantic predators however have yet to fully recognize the lionfish as prey and attempts to “train” them by feeding them captured lionfish had instead encouraged them to approach divers more boldly. In some cases, this led to increased aggression and even forced the need for a dedicated lookout diver to watch for approaching predators during hunting dives [5].

Currently the most effective method for controlling lionfish populations is manual removal of the invasive species from affected areas. This is primarily done by spearfishing. This approach, despite its proven effectiveness, does still have some shortcomings. Spearfishing near reefs, where lionfish often congregate, is prohibited by some governments due to the potential damage it could inflict on corals and other protected marine life. Additionally, the effectiveness of manual lionfish removal is dependent on how many divers are available. These divers are often volunteers, not registered workers, so their availability for conservation efforts can be uncertain at times. Another potential issue stemming from this reliance on volunteers is that these are usually recreational divers. Lionfish will sometimes reside in areas that may be too dangerous for the average diver to reach and thus require the need for professional divers or expensive diving equipment [3].

1.2 Objectives

Having identified the most effective method for controlling the lionfish populations, this project aimed to create a new tool that could potentially increase the efficiency of this method while also minimizing the aforementioned shortcomings. Creating a model capable of targeting lionfish while also identifying native species as avoidable would hopefully pave the way for automizing lionfish scouting and hunting in the future. It was vital that this model be as accurate as possible since misidentifying lionfish too often could lead to native animals being targeted, which defeats the purpose of protecting them and their environment. Too many identification errors could also lead to needlessly expending human workers to areas where they may not have been needed. Available personnel are already stretched thin as is [3].

1.3 Achievements

Using datasets of aquatic life alongside images of lionfish, a convolutional neural network capable of binary classification was successfully trained. This model accepts images as input and identifies if they do or do not contain lionfish with high accuracy.

1.4 Overview of Dissertation

This dissertation will briefly describe previous research that was done on the issue of invasive lionfish as well as looking at other machine learning models that have been made on this topic. This includes highlighting their strengths and exploring how they address issues with current methods of lionfish hunting.

After discussing previous works done on this topic, the process of creating this new model will be described. This includes following the steps in the methodology, which were based on the standard practices used in CRISP-DM. At each step, the methods and techniques used will be discussed and, if necessary, background information will be provided for context. Choices made and the reasoning behind them will be explained. Errors and their corrections will be highlighted as well.

Once the creation process for the model has been explained, the results of the training will be shown and explained further. The overall performance of the model will be evaluated and conclusions will be drawn. Potential areas for improvement as well as further research in the future will be explored.

2 Literature Review

The study of invasive lionfish has been ongoing for quite some time and much research has gone into trying to control their populations. As was mentioned previously, the most effective method of doing this thus far has been manually removing lionfish from the environments they are invading [3]. The idea of integrating machine learning into lionfish hunting has been explored a few times before.

A team of researchers created a lionfish-based model capable of object detection. It was trained using images and videos of lionfish. When presented with imagery of the target species, this model would highlight where on the image it thinks it detects a lionfish and then give a confidence score. The overall accuracy of this model ranged in the 80% range. The researchers reflected on the performance of the model and noted that it sometimes failed to detect lionfish if there was too much visual clutter. The model also occasionally misidentified other animals as lionfish due to similar shapes, colours or patterns [6]. This misidentification issue was brought up earlier as a concern related to lionfish hunting: The accidental harming of native animals due to errors [4]. It is likely that this was a result of exclusively using lionfish images to train the model. This dissertation project sought to avoid this issue by using an additional dataset during training which contained images of other aquatic animals. This way, the new model would learn to both recognize lionfish but also recognize when something was explicitly not a lionfish.

Another similar model was tested by a different research team, also with a focus on object detection. This model similarly worked by projecting a confidence level whenever it detected a lionfish, however this one was built to reject a lionfish detection if the confidence level was below 80%. Like the previous model, this algorithm was exclusively trained on lionfish images and also experienced issues with misidentifying other objects and animals as lionfish if they had similar features or colours to the target species. This model was around 90% accurate with its detections, which was an improvement over the last model. The authors specified that they expected this model to be supervised by a person, who could then manually confirm if the lionfish detections were accurate [7]. While understandable, the need for a person to operate the model somewhat goes against one of the goals for this project, which was to address limited personnel by lowering the need for actual human labour. Instead, the model made for this dissertation sought to improve further on the accuracy score, thus lowering the number of misidentifications altogether. The misidentification issues again highlight the need to include other animals in the training process to ensure the model will avoid targeting native species.

Though these projects used different systems and coding languages than the ones chosen for this project, the insights they shared into their training methods and the observations made during their testing helped guide some of the decisions made during this dissertation.

3 Methodology

Early on in the project, it was already decided that the topic would be tied to the Caribbean region in some way or involve animals. Choosing invasive lionfish essentially combines both ideas along with adding a new facet to the project: environmental protection. While researching the chosen topic, other models based on the same topic were discovered. Though still helpful, these models did not address some previous concerns raised in regards to lionfish hunting [3][4].

3.1 Project Planning

Observing previous models, they seemed to face issues with their algorithms misidentifying native animals with similar shapes, patterns and colours as lionfish. This newer model sought to improve on this weakness by including images of other aquatic animals into the training process. This would allow the model to not only recognize lionfish, but also identify when something was explicitly *not* a lionfish. Previous models managed to achieve accuracy scores in the 80-90% range, so this model sought to improve on that. A more accurate model could better assist with scouting lionfish populations, allowing hunting groups to divide their workers more efficiently to deal with areas that may be more or less impacted by lionfish. This would hopefully alleviate the issue of limited volunteers for conservation efforts [3].

Python was chosen as the coding language for this project since it hosts various libraries such as TensorFlow which are quite adept at creating neural networks. Tests would be carried out in both IDLE and Jupyter Notebooks.

3.2 Collecting Data

A dataset containing various images of lionfish was found on Kaggle. It contained 955 images of lionfish. Two other additional datasets from that same source were also collected. These contained images of various aquatic animals. These were designated to be used for the “not lionfish” dataset. These images would all need to be processed before the model could use them [8].

Some potential issues with the data were identified:

- The additional datasets for aquatic life also contained images of lionfish, which would likely confuse the model.
- When the data was added together and then divided into two groups (lionfish and non-lionfish), the datasets were slightly unbalanced.
- The images were not of equal size.

3.3 Data Preparation

The issues identified during the data analysis needed to be addressed before training could start. The aquatic life datasets were inspected and any images of lionfish were removed and added to the other dataset instead. After this was done, both aquatic life datasets were combined into a single dataset: non-lionfish.

The lionfish and non-lionfish datasets were slightly unbalanced however: the lionfish dataset contained 999 images, while the non-lionfish dataset contained 947 images. While this was likely not a large enough difference to cause major problems, it was decided that class weights would be implemented as a precaution. This was to ensure that the model did not become biased towards the larger dataset [9]. Scikit-Learn has built in functions available to address this. These will be explored further in the model building section.

The images from the original lionfish dataset were larger than the images in the aquatic life datasets. Image quality is rapidly lost when images are made larger. Since it was important to maintain image clarity for the model, 256x256 was chosen as a compromise size. This size meant that the largest images would be made smaller while the smallest images would only need to be slightly increased in size, thus preserving the most overall image quality [10]. Using the specified image size alongside the image generator function from TensorFlow allowed the code to normalize the size of all images before they were uploaded to the model.

3.4 Model Building

The two necessary datasets were created in the previous step, however they now needed to be split into test, training and validation sets. This was done by installing splitfolders and then running a Python script in IDLE. This randomized the images and divided them into the specified ratios [11]. Initially a test/train/validation split of 80/10/10% was chosen, as this is a common standard in machine learning. After some test runs however, the results seemed to imply that the test and validation sets were potentially too small. This made it difficult to properly assess how the model was performing. Later versions of the model were instead training on data split ratio of 70/15/15%. This change seemed to provide more promising results and was kept for the final model [12].

With the data sets split, the actual coding of the model could begin. The necessary libraries (OS, NumPy, TensorFlow, Matplotlib and SKLearn) were imported. To ensure consistent results and reproducibility, a random seed value was added. Paths to the necessary datasets were defined. As the data split was done manually before the model was trained, paths to each of these datasets also needed to be defined [13].

As was decided previously, the image sizes were defined (256x256) and a batch size of 32 was chosen. This batch size allowed for a decent number of images to be processed without drastically overworking the CPU. With the chosen sizes defined, the appropriate functions were then called to normalize the images and label them. A TensorFlow dataset was then created to store this processed data [14]. The shuffle function was called to ensure randomization of data during the training process. This was to avoid the model potentially learning from the order of the images rather than their content [15].

The initial runs of the model encountered some issues during training related to data shortages. This was due to using the image generator function and was resolved by instead using the load and preprocess image function. This allowed the use of the repeat function to address issues of data shortage during training. Batch and prefetch functions were called to make the loading process more efficient. The number of steps per epoch were calculated automatically in the code to ensure the most efficient values based on the data [16].

As was discussed in the previous section, class weights were included in the code to address a slight imbalance in the datasets. These were calculated automatically in the code based on the number of images in both datasets. This ensured that the model did not become biased towards the larger dataset. Since this model contained multiple classes, the calculated class weights were converted into a dictionary format. This allowed different weights to be assigned to different classes [9].

Finally, the Sequential model was built and the layers were defined. These layers were:

- **Input Layer:** This defined the size and shape of the images (256x256).
- **Conv2D Layers:** These layers applied filters over the images at size 3x3 (effective size while still maintaining efficiency) with activation function ReLU (introduced non-linearity to the model, allowing it to learn more complex patterns). As the model progressed, more filters would be applied. The first Conv2D applied 32 filters, the second applied 64 and the third layer applied 128 layers – each capturing more complex and abstract patterns.
- **Maxpooling2D Layers:** This layer reduced spatial dimensions. This made the model perform more efficiently while also improving its ability to detect features even when they changed positions. One of these layers was applied after every Conv2D layer.
- **Flatten Layer:** This converted the 2D data into 1D vectors. This was to prepare the data for the next layer, which only accepted 1D inputs.
- **1st Dense Layer:** This layer contained 128 neurons (enough to perform complex tasks without being overly demanding on the CPU or risking overfitting the model) and integrated all the information collected by the previous layers. Activation function ReLU was used once more to introduce non-linearity.
- **Dropout Layer:** This was a regularization layer that would randomly drop some units during each training step. This taught the model not to become too dependent on singular features or specific patterns, thus preventing overfitting.
- **2nd Dense Layer:** The final output layer. Since this output would be a binary classification, only 1 neuron was needed and the activation function sigmoid was used. Sigmoid again introduces non-linearity to the model while also being particularly well suited for binary classification.

After establishing the layers of the neural network, the model was compiled. The chosen optimizer was Adam since it is generally reliable while also having built in features that correct potential biases that the model may form during training. Binary Cross-Entropy was chosen for the loss function since this model was dealing with a binary classification problem. Similarly to the Sigmoid function chosen earlier, this function was especially well suited to binary classification. For metrics, accuracy was chosen as this was important to the goals established early on for this project [17].

Early stopping was implemented into the code. If the accuracy of the model did not improve after 3 epochs, the training would be halted early to ensure the model did not become overfitted and the best recorded weights would be restored [18].

When it was time to train the model, the maximum number of epochs chosen was 30. This was done to give the model sufficient time to train properly without making the waiting time for results overly long, granted this was not much of an issue due to the earlier implementation of early stopping [19]. Very rarely did the model need more than 20 epochs before the performance scores began to plateau, typically stopping training at around 10 to 15 epochs.

Lastly, to help with evaluation of the model, the code would print a classification report, showing various results from the training such as loss, accuracy, precision, recall and f1 scores. Graphs were also printed to show the models progress during training, as well as a confusion matrix showing its performance on the test data [14].

4 Conclusion

4.1 Summary

The final model was a convolutional neural network able to perform binary classification when presented with images of aquatic life, identifying them as either “Lionfish” or “Not Lionfish” with roughly 95% accuracy.

4.2 Evaluation

The final model printed a classification report and a confusion matrix to show its performance on the test data (see Figure 1 in Appendix).

The test data contained 294 images total. These were 151 images of lionfish and 143 images of other aquatic life. The report showed that the model was able to correctly identify 95% of the lionfish images as “Lion Fish” and 97% of the other aquatic life images were correctly identified as being “Not Lionfish”.

Both classes achieved an F1-score of 0.96, showing a good balance between precision and recall for both. This implied that the model was not overly biased towards choosing one class over the other [20]. This was a potential concern earlier in the data preparation phase due to slightly unbalanced datasets however this was mitigated by implementing class weights in the code.

The model achieved an accuracy score of 96%. As was mentioned earlier in the project planning section, it was vital for this project that the model achieve a high accuracy score as this would minimize instances of native wildlife being targeted accidentally or divers receiving false alarms.

The confusion matrix showed that out of 294 images, the model correctly identified 283 of them, but misidentified 11 of them. Though not perfect, this was still a very acceptable result.

The final model also printed two graphs showing its learning curve during the training process (see Figure 2 in Appendix).

The accuracy graph showed steady improvement for both the training and validation graphs. The shape of the graphs implied that both were close to reaching a plateau but the early stopping function ended the epochs early to prevent the model from potentially become worse.

The loss graph showed both lines rapidly decreasing before becoming somewhat more stable, which was desirable. However, a better fitting model would have a very small gap between the training and validation lines. This graph seemed to show a wider gap between them, which implied that the datasets used might have been too small [21].

4.3 Future Work

As was implied in the loss graph, this project potentially had too little data to create a better fitting model. Regrettably, this was not an issue that could be tackled with the resources available during the project. In the future, this project could be improved with both larger datasets as well as a larger variety of aquatic life images to ensure the model is truly able to recognize the differences between classes.

Observing the difficulties experienced with the other lionfish detection models discussed in the literature review, this model aimed to minimize the risk of misidentification and included images of other aquatic animals. The final model was able to achieve a slightly higher accuracy score compared to the other models which relied exclusively on lionfish imagery for training.

In the future, if larger and more diverse marine life datasets become widely available for public use, this model could be improved upon, perhaps even working in combination with the object detection models that were developed previously. This model would assist in minimizing the misidentification issues that the other models struggled with.

Once a sufficiently accurate model can be developed and implemented, the demand for volunteer divers could be lessened by instead using drones to identify where lionfish are. This would allow organizations to divide their members more efficiently to other areas of lionfish control without exhausting their limited numbers.

This model could also be expanded into other areas of conservation. With sufficient data to train it, it could easily be retrofitted to detect other invasive species besides just lionfish. It could instead also be used to identify endangered species in the wild to monitor population sizes.

References

- [1] NOAA, "Impacts of Invasive Lionfish," NOAA, Jul. 29, 2022. <https://www.fisheries.noaa.gov/southeast/ecosystems/impacts-invasive-lionfish>
- [2] M. Brown, "Study shows most efficient ways to capture invasive lionfish," *phys.org*, Jan. 10, 2022. <https://phys.org/news/2022-01-efficient-ways-capture-invasive-lionfish.html>
- [3] M. Vermeij, "The Effectiveness of Volunteer Lionfish Removal Efforts," *DCNA*, Aug. 08, 2013. <https://dcnanature.org/the-effectiveness-of-volunteer-lionfish-removal-efforts/>
- [4] V. J. Giglio, O. J. Luiz, M. C. Barbosa, and C. E. L. Ferreira, "Behaviour of recreational spearfishers and its impacts on corals," *Aquatic Conservation: Marine and Freshwater Ecosystems*, vol. 28, no. 1, pp. 167–174, Jul. 2017, doi: <https://doi.org/10.1002/aqc.2797>.
- [5] S. Murray, "A new study outlines the do's and don'ts of managing invasive lionfish," *News*, May 24, 2022. <https://blogs.ifas.ufl.edu/news/2022/05/24/a-new-study-outlines-the-dos-and-donts-of-managing-invasive-lionfish/>
- [6] A. T. Martínez-González, V. M. Ramírez-Rivera, J. A. Caballero-Vázquez, D. A. G. Jáuregui, "Deep learning algorithm as a strategy for detection an invasive species in uncontrolled environment," *ResearchGate*, Sept. 5, 2021. https://www.researchgate.net/publication/354378322_Deep-learning-algorithm-as-a-strategy-for-detection-an-invasive-species-in-uncontrolled-environment#full-text
- [7] M. M. N. Shargh, H. Myler, H. Zargarzadeh "Design and Implementation of an Assistive Real-Time Red Lionfish Detection System for AUV/ROVs", *Wiley Online Library*, Nov. 11, 2018. <https://onlinelibrary.wiley.com/doi/10.1155/2018/5298294>
- [8] TensorFlow, "Load Images with tf.data | TensorFlow Core," *TensorFlow*, 2019. https://www.tensorflow.org/tutorials/load_data/images
- [9] K. Singh, "How to Improve Class Imbalance using Class Weights in Machine Learning," *Analytics Vidhya*, Oct. 06, 2020. <https://www.analyticsvidhya.com/blog/2020/10/improve-class-imbalance-class-weights/>
- [10] J. Nelson, "You might be resizing your images incorrectly.," *Roboflow Blog*, Jan. 31, 2020. <https://blog.roboflow.com/you-might-be-resizing-your-images-incorrectly/>
- [11] J. Filter, "split-folders: Split folders with files (e.g. images) into training, validation and test (dataset) folders.," *PyPI*. <https://pypi.org/project/split-folders/>
- [12] V. R. Joseph, "Optimal ratio for data splitting," *Statistical Analysis and Data Mining: The ASA Data Science Journal*, vol. 15, no. 4, pp. 531–538, Apr. 2022, doi: <https://doi.org/10.1002/sam.11583>
- [13] M. Tripathi, "Image Processing using CNN | Beginner's Guide to Image Processing," *Analytics Vidhya*, Jun. 21, 2021. <https://www.analyticsvidhya.com/blog/2021/06/image-processing-using-cnn-a-beginners-guide/>
- [14] D. Mwit, "How to build CNN in TensorFlow: examples, code and notebooks | cnvrg.io," Feb. 23, 2021. <https://cnvrg.io/cnn-tensorflow/>
- [15] S. Dutta, "What is Shuffling the Data? A Guide for Students - Sanjay Dutta - Medium," *Medium*, Jul. 16, 2024. https://medium.com/@sanjay_dutta/what-is-shuffling-the-data-a-guide-for-students-0f874572baf6
- [16] Stack Overflow, "When is .repeat used when loading a Tensorflow dataset for training?," *Stack Overflow*, 2024. <https://stackoverflow.com/questions/69281548/when-is-repeat-used-when-loading-a-tensorflow-dataset-for-training>
- [17] M. Gurucharan, "Basic CNN Architecture: Explaining 5 Layers of Convolutional Neural Network," *upGrad blog*, Dec. 07, 2020. <https://www.upgrad.com/blog/basic-cnn-architecture/>

- [18] J. Brownlee, "Use Early Stopping to Halt the Training of Neural Networks At the Right Time," *Machine Learning Mastery*, Dec. 09, 2018.
<https://machinelearningmastery.com/how-to-stop-training-deep-neural-networks-at-the-right-time-using-early-stopping/>
- [19] GeeksforGeeks, "How to choose Batch Size and Number of Epochs When Fitting a Model?," *GeeksforGeeks*, Jul. 10, 2024. <https://www.geeksforgeeks.org/how-to-choose-batch-size-and-number-of-epochs-when-fitting-a-model/>
- [20] N. Sharma, "Understanding and Applying F1 Score: A Deep Dive with Hands-On Coding," *Arize AI*. <https://arize.com/blog-course/f1-score/#:~:text=F1%20score%20computes%20the%20average>
- [21] J. Brownlee, "A Gentle Introduction to Learning Curves for Diagnosing Machine Learning Model Performance," *Machine Learning Mastery*, Apr. 03, 2019.
<https://machinelearningmastery.com/learning-curves-for-diagnosing-machine-learning-model-performance/>

Appendix

Test Accuracy: 0.9626
10/10 ————— 3s 265ms/step
Classification Report:

	precision	recall	f1-score	support
Not Lionfish	0.97	0.95	0.96	151
Lionfish	0.95	0.97	0.96	143
accuracy			0.96	294
macro avg	0.96	0.96	0.96	294
weighted avg	0.96	0.96	0.96	294

Confusion Matrix:
[[144 7]
[4 139]]
Precision: 0.9521
Recall: 0.9720
F1 Score: 0.9619

Figure 1. Classification Report and Confusion Matrix

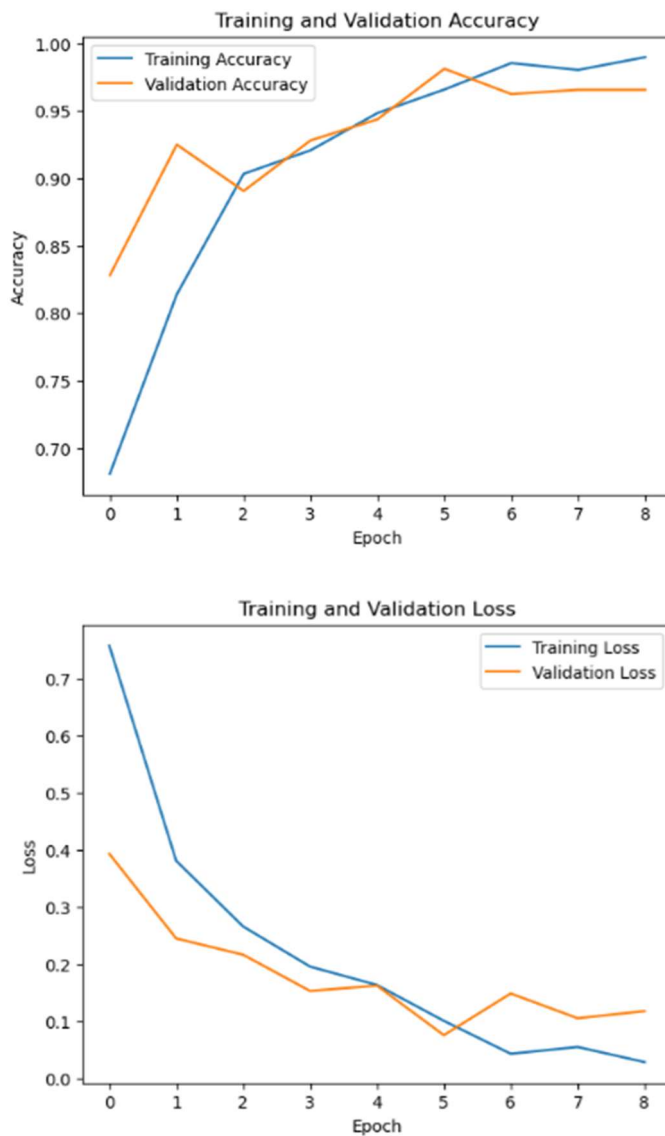


Figure 2. Training and Validation Graphs